

[FORMATION INTENSIVE]

CTF RÉSEAU

Entraînement Intensif 10 Jours

■ Wireshark & Filtres avancés	Jour 1
■ Protocoles TCP/IP en profondeur	Jour 2
■ Sniffing & Traffic Analysis	Jour 3
■ Crypto réseau & TLS	Jour 4
■ Firewall, IDS, Pivoting	Jour 5
■ Forensics PCAP CTF	Jour 6
■ Protocoles obscurs & Tunneling	Jour 7
■ Scripting réseau (Scapy/Python)	Jour 8
■ CTF Platforms Sprint	Jour 9
■ Simulation compétition complète	Jour 10

Objectif : Passer de débutant/intermédiaire à un niveau compétitif CTF réseau en 10 jours. Chaque jour = 1 thème maîtrisé + plateformes pratiques recommandées. Ce document est conçu pour comprendre *comment ça marche*, pas pour mémoriser des commandes.

TABLE DES MATIÈRES

JOUR 1	Maîtriser Wireshark — Filtres, Dissection, Analyse	3
JOUR 2	Protocoles TCP/IP — Fonctionnement interne profond	8
JOUR 3	Sniffing & Traffic Analysis avancée	11
JOUR 4	Cryptographie réseau — TLS/SSL, SSH, VPN	14
JOUR 5	Firewall, IDS/IPS, Pivoting réseau	17
JOUR 6	Forensics PCAP — Méthode CTF complète	20
JOUR 7	Protocoles exotiques & Covert Channels	23
JOUR 8	Scripting réseau — Scapy & Python	26
JOUR 9	Sprint CTF Platforms	29
JOUR 10	Simulation compétition complète	31

JOUR 1

MAÎTRISER WIRESHARK — Filtres, Dissection & Analyse

Philosophie du jour : Wireshark n'est pas une liste de commandes à mémoriser. C'est un **langage de requête** sur des structures de données. Si tu comprends comment les paquets sont structurés en couches, tu peux écrire n'importe quel filtre sans jamais l'avoir vu.

1.1 — Comment Wireshark voit le monde

Wireshark capture et **disséque** chaque paquet en arbre de champs. Chaque protocole expose ses champs sous la forme **protocole.champ**. C'est la clé de tout : les filtres ne sont que des expressions booléennes sur ces champs.

La structure d'un paquet vue par Wireshark

Quand un paquet arrive, Wireshark applique ses *dissecteurs* de couche en couche :

Couche 2 — Ethernet	eth	eth.src, eth.dst, eth.type
Couche 3 — IP	ip	ip.src, ip.dst, ip.proto, ip.ttl, ip.flags
Couche 4 — TCP/UDP	tcp / udp	tcp.srcport, tcp.dstport, tcp.flags, tcp.seq, tcp.ack
Couche 7 — HTTP	http	http.request.method, http.host, http.response.code
Couche 7 — DNS	dns	dns.qry.name, dns.resp.addr, dns.flags.response
Couche 7 — TLS	tls	tls.handshake.type, tls.record.version

1.2 — Les deux types de filtres (fondamental !)

Il existe **deux systèmes de filtres totalement différents** dans Wireshark. Confondre les deux est l'erreur la plus commune.

Capture Filter (BPF)	Display Filter (DFT)
S'applique avant la capture. Syntaxe Berkeley Packet Filter (BPF).	S'applique après la capture pour afficher/masquer. Syntaxe Wireshark propriétaire.
<pre>host 192.168.1.1 port 80 tcp and port 443</pre>	<pre>ip.addr == 192.168.1.1 tcp.port == 80 tcp && tcp.port == 443</pre>
Moins expressif, moins de champs disponibles.	Très expressif, accès à tous les champs de tous les protocoles.

1.3 — La grammaire des Display Filters : construire, pas mémoriser

Un display filter est une **expression booléenne**. Voici sa grammaire complète — si tu la comprends, tu peux construire n'importe quel filtre :

Opérateurs de comparaison

```
== ou eq égal à
!= ou ne différent de
> ou gt supérieur à
< ou lt inférieur à
>= ou ge supérieur ou égal
<= ou le inférieur ou égal
contains le champ contient la valeur (substring)
matches correspond à une regex (Perl-compatible)
in appartient à un ensemble ex: tcp.port in {80 443 8080}
```

Opérateurs booléens

```
&& ou and ET logique
|| ou or OU logique
! ou not Négation
() Groupement (priorité)
```

Opérateurs sur les bits (réseau/masques)

```
& ET bit-à-bit ex: ip.flags & 0x02 (bit DF set)
| OU bit-à-bit
^ XOR
```

Sélection de sous-champs et slicing

Tu peux accéder à des octets précis d'un champ avec la notation **champ[offset:longueur]** :

```
tcp.payload[0:2] -- 2 premiers octets du payload TCP
eth.src[0:3] -- OUI (Organizationally Unique Identifier) de l'adresse MAC
frame[24:4] -- 4 octets à offset 24 dans le frame brut
```

■ COMPRENDRE, PAS MEMORISER

La logique est simple : **protocole.champ opérateur valeur**. Pour connaître le nom d'un champ, il suffit de **cliquer dessus dans le panneau de dissection** : Wireshark affiche le nom exact dans la barre de statut. Tu peux ensuite le copier dans le filtre avec clic droit → "Apply as Filter". C'est ainsi que les pros construisent leurs filtres.

1.4 — Filtres essentiels CTF par catégorie

■ Identification du trafic

```
-- Par adresse IP
ip.addr == 10.0.0.1 -- src OU dst = cette IP
ip.src == 10.0.0.1 -- source uniquement
ip.dst == 10.0.0.1 -- destination uniquement
-- Par réseau (CIDR)
ip.addr == 192.168.0.0/16 -- tout le réseau 192.168.x.x
```

```
-- Par port
tcp.port == 4444 -- src ou dst port 4444 (reverse shell classique)
tcp.dstport in {21 22 23 25 80 443 445 3389} -- ports courants

-- Par protocole
http or dns or ftp -- trafic HTTP, DNS ou FTP
not arp and not icmp -- exclure le bruit réseau
```

■ Recherche de données / flags CTF

```
-- Chercher une string dans tout le trafic
frame contains "flag"
frame contains "CTF{"
frame contains "password"

-- Dans un protocole spécifique
http contains "flag"
dns.qry.name contains "flag"
ftp-data contains "flag"

-- Regex (puissant !)
http.request.uri matches ".*flag.*"
http matches "[Pp]assword"
frame matches "flag\[0-9a-f]+\\"
```

■ Détection d'anomalies et comportements suspects

```
-- Scans de ports (TCP SYN sans ACK = SYN scan)
tcp.flags.syn == 1 and tcp.flags.ack == 0

-- Connexions RST (refusées / scan)
tcp.flags.reset == 1

-- Trafic exfiltration DNS (noms longs)
dns.qry.name.len > 50

-- HTTP non-standard (port bizarre)
http and tcp.dstport != 80 and tcp.dstport != 8080 and tcp.dstport != 443

-- TTL anormalement bas (traceroute, evasion)
ip.ttl < 10

-- Fragmentation IP
ip.flags.mf == 1 or ip.frag_offset > 0

-- Trafic TLS avec SNI suspect
tls.handshake.extensions_server_name contains "dyn"
```

■ Reconstruction de fichiers / sessions

```
-- Suivi d'une conversation TCP spécifique
tcp.stream eq 0 -- stream numéro 0
tcp.stream eq 5 -- stream numéro 5

-- Filtrer une session complète (après avoir cliqué sur un paquet)
-- Clic droit > Follow > TCP Stream (Wireshark génère le filtre automatiquement)

-- Transferts FTP data
ftp-data -- le contenu des fichiers transférés

-- SMB file transfers
```

```
smb2.cmd == 9 -- SMB2 READ
smb2.filename contains "flag"
```

1.5 — Features avancées de Wireshark

Statistics → Conversations & Endpoints

Ces deux vues te donnent une **vue d'ensemble immédiate** du trafic : qui parle à qui, quel volume. C'est ton premier réflexe dans un CTF PCAP. Tu identifies en 10 secondes les paires IP anormales, les ports inhabituels, et les transferts de données importants.

Statistics → Protocol Hierarchy

Affiche tous les protocoles présents dans la capture avec leur pourcentage. Si tu vois "Data" à 40% et que c'est un challenge, c'est que des données sont transférées en clair ou dans un protocole custom — cherche là.

File → Export Objects

Wireshark peut extraire automatiquement les fichiers transférés via HTTP, SMB, FTP, DICOM, TFTP. Dans un CTF, **toujours** aller voir Export Objects → HTTP en premier.

Analyze → Follow Stream

Follow TCP/UDP/HTTP/TLS Stream reconstitue la conversation complète entre deux hôtes. C'est essentiel pour lire une session HTTP, FTP, SMTP en clair.

Coloring Rules

Wireshark colore automatiquement le trafic (rouge = erreurs TCP, vert = HTTP, etc.). Tu peux créer tes propres règles : View → Coloring Rules. Utile pour surligner les streams suspects.

tshark — Wireshark en ligne de commande

tshark est la version CLI de Wireshark. Dans un CTF, il permet de scripter l'analyse :

```
# Lire un pcap et filtrer
tshark -r capture.pcap -Y "http.request" -T fields -e ip.src -e http.request.uri

# Extraire tous les objets HTTP
tshark -r capture.pcap --export-objects http, ./output/

# Statistiques des conversations
tshark -r capture.pcap -q -z conv,tcp

# Chercher un pattern dans les données
tshark -r capture.pcap -Y "frame contains \"flag\"" -T fields -e frame.number -e ip.src

# Décoder un protocole custom sur un port
tshark -r capture.pcap -d tcp.port==4444,http
```

1.6 — Méthodologie CTF Wireshark (workflow)

1 Vue d'ensemble

Statistics → Protocol Hierarchy + Conversations. Identifie les protocoles, le volume, les IPs anormales.

2 Export Objects

File → Export Objects → HTTP (et SMB si présent). Recherche des fichiers suspects.

3 Strings / flags

frame contains "flag" ou frame contains "CTF". Recherche directement.

4 Follow Streams

Clique sur les streams suspects, Follow TCP/HTTP Stream. Lis le contenu.

- | | | |
|----------|-------------------------------|---|
| 5 | Protocoles inhabituels | Protocol Hierarchy : tout ce qui est rare mérite attention. DNS avec gros volume ? ICMP data ? Regarde. |
| 6 | Analyse fine | Pour les challenges complexes : filtres sur les flags TCP, TTL, timestamps, séquences anormales. |

■ PLATEFORMES PRATIQUES — JOUR 1

- **PicoCTF** (picoctf.org) — Cherche les challenges "Wireshark o wireshark" et "shark on wire"
- **CyberDefenders** (cyberdefenders.org) — Labs PCAP dédiés réseau, très bien faits
- **Hack The Box — Sherlock** (hackthebox.com) — Challenges forensics réseau
- **PacketLife.net** — Télécharge des captures de référence pour t'entraîner
- **CloudShark** (cloudshark.org) — Captures publiques annotées

JOUR 2

PROTOCOLES TCP/IP — Fonctionnement interne

Philosophie du jour : Un CTF réseau teste ta capacité à identifier des **anomalies**. Pour voir une anomalie, tu dois connaître le comportement normal. Ce jour est consacré à comprendre ce qui se passe *réellement* dans chaque protocole, octet par octet si nécessaire.

2.1 — TCP : le handshake et les flags

TCP utilise **8 flags** dans l'entête. Comprendre leur combinaison te permet de reconstruire l'état de n'importe quelle connexion.

Flag	Hex	Rôle	Filtre Wireshark
SYN	0x02	Initiation connexion	tcp.flags.syn == 1
ACK	0x10	Accusé de réception	tcp.flags.ack == 1
FIN	0x01	Fin connexion (propre)	tcp.flags.fin == 1
RST	0x04	Reset (forcé / erreur)	tcp.flags.reset == 1
PSH	0x08	Push data immédiatement	tcp.flags.push == 1
URG	0x20	Données urgentes	tcp.flags.urg == 1
ECE	0x40	ECN Echo	tcp.flags.ecn == 1
CWR	0x80	Congestion Window Reduced	tcp.flags.cwr == 1

Le 3-way handshake TCP — ce qui est normal

```
Client → Serveur : SYN (flags: 0x002)
Serveur → Client : SYN-ACK (flags: 0x012)
Client → Serveur : ACK (flags: 0x010)
--- connexion établie ---
Données échangées...
Client → Serveur : FIN-ACK (flags: 0x011)
Serveur → Client : FIN-ACK (flags: 0x011)
Client → Serveur : ACK (flags: 0x010)
```

Anomalies TCP courantes en CTF

```
-- SYN scan Nmap : SYN sans jamais de ACK complet
tcp.flags == 0x002 and not tcp.analysis.retransmission
-- Connexion réinitialisée immédiatement (port fermé)
tcp.flags.reset == 1
-- Null scan (aucun flag)
tcp.flags == 0x000
-- Xmas scan (FIN+PSH+URG)
```

```
tcp.flags == 0x029
-- Données dans le paquet SYN (rare, exfiltration)
tcp.flags.syn == 1 and tcp.payload
```

2.2 — IP : TTL, fragmentation, options

L'IP Time To Live (TTL) est souvent un indice sur l'OS source. Les valeurs initiales par défaut sont :

```
Linux / Android : 64
Windows : 128
Cisco / réseau : 255
macOS / iOS : 64 (ou 255 pour certains services)
```

Un TTL qui arrive à destination avec une valeur bizarre peut indiquer un nombre de sauts anormal ou une manipulation du paquet.

Fragmentation IP

IP peut fragmenter les paquets trop grands pour traverser un lien. Chaque fragment a un **Fragment Offset**. En CTF, la fragmentation peut cacher des données ou contourner des IDS.

```
-- Voir tous les fragments
ip.flags.mf == 1 -- More Fragments : pas le dernier fragment
ip.frag_offset > 0 -- Ce n'est PAS le premier fragment
-- Chevauchement de fragments (évasion IDS)
-- Nécessite une analyse manuelle ou des outils spécialisés
```

2.3 — DNS en profondeur

DNS est **extrêmement** utilisé en CTF pour le tunneling et l'exfiltration. Un attaquant peut encoder des données dans les noms de domaine DNS et les transmettre sans ouvrir de connexion TCP.

Types d'enregistrements DNS à connaître

```
A : IPv4 dns.qry.type == 1
AAAA : IPv6 dns.qry.type == 28
MX : Mail exchanger dns.qry.type == 15
TXT : Texte libre dns.qry.type == 16 <-- très utilisé pour exfil
CNAME : Alias dns.qry.type == 5
PTR : Reverse lookup dns.qry.type == 12
NS : Name server dns.qry.type == 2
```

Détection DNS Tunneling

```
-- Noms très longs (données encodées base64)
dns.qry.name.len > 50
-- Beaucoup de requêtes vers un seul domaine
dns.qry.name contains "attacker.com"
-- Requêtes TXT (exfil dans les réponses)
dns.qry.type == 16
-- Entropie élevée dans le nom (base64/hex)
-- Utilise tshark + scripts pour calculer l'entropie
```

```
tshark -r cap.pcap -Y dns -T fields -e dns.qry.name | sort -u
```

2.4 — HTTP/HTTPS : ce que tu dois chercher

```
-- Méthodes HTTP
http.request.method == "GET"
http.request.method == "POST"
http.request.method == "PUT"

-- Codes de réponse
http.response.code == 200 -- OK
http.response.code == 302 -- Redirection
http.response.code == 404 -- Not found
http.response.code >= 500 -- Erreurs serveur

-- Headers intéressants
http.request.uri contains "admin"
http.cookie contains "session"
http.authorization -- HTTP Basic Auth (credentials en base64!)

-- Content type
http.content_type contains "multipart" -- upload de fichier
http.content_type == "application/json"
```

■ PLATEFORMES PRATIQUES — JOUR 2

- **CyberDefenders — PacketMaze** : challenge TCP/DNS avancé
- **TryHackMe — Wireshark: The Basics** et **Wireshark: Traffic Analysis**
- **PicoCTF** — Challenges "whats-going-on", "tunn3l v1s10n"
- Crée ta propre capture : lance un scan Nmap contre une VM, capture avec Wireshark, identifie les patterns

JOUR 3

SNIFFING & TRAFFIC ANALYSIS AVANCÉE

3.1 — ICMP : pas qu'un simple ping

ICMP est souvent ignoré mais c'est un vecteur d'exfiltration classique. Un attaquant peut cacher des données dans le **payload ICMP** (la partie "data" du ping).

```
-- Types ICMP importants
icmp.type == 8 -- Echo Request (ping aller)
icmp.type == 0 -- Echo Reply (pong)
icmp.type == 3 -- Destination Unreachable
icmp.type == 11 -- Time Exceeded (traceroute)

-- Détecter un payload ICMP anormal (ICMP tunneling)
icmp.type == 8 and data.len > 64 -- payload > 64 octets = suspect
icmp -- voir tous les ICMP, chercher du texte dans le payload
```

3.2 — ARP : qui es-tu vraiment ?

ARP est vulnérable car il est **sans authentification**. L'ARP poisoning est une attaque MITM classique. En CTF forensics, on te demande souvent d'identifier un ARP poisoning dans une capture.

```
-- Voir tout le trafic ARP
arp

-- Requêtes ARP "qui a cette IP ?"
arp.opcode == 1 -- ARP Request
arp.opcode == 2 -- ARP Reply

-- Détecter ARP poisoning : une IP mappée à plusieurs MAC
-- chercher des réponses ARP non sollicitées (Gratuitous ARP)
arp.opcode == 2 and arp.src.proto_ipv4 == arp.dst.proto_ipv4

-- En pratique : Statistics > Endpoints > Ethernet
-- Puis filtrer : arp.src.hw_mac == [mac_suspect]
```

3.3 — Protocoles d'authentification en clair

Beaucoup de challenges CTF consistent à récupérer des credentials depuis un PCAP. Voici les protocoles où les mots de passe passent en clair :

FTP	<code>ftp.request.command == "PASS"</code>	Le mot de passe est dans la commande PASS. Follow Stream pour voir tout.
Telnet	<code>telnet</code>	Chaque touche tapée = un paquet. Follow TCP Stream pour reconstituer la session.
HTTP Basic	<code>http.authorization</code>	Base64 décodable : <code>echo 'dXNlcjpwYXNz' base64 -d</code>

SMTP	<code>smtp.req.command == "AUTH"</code>	Credentials en base64 après AUTH PLAIN ou AUTH LOGIN.
POP3	<code>pop.request.command == "PASS"</code>	Mot de passe en clair dans la commande PASS.
IMAP	<code>imap contains "LOGIN"</code>	Identifiants après la commande LOGIN.
SMB/NTLMv2	<code>ntlmssp.auth.username</code>	Hash NTLMv2 crackable avec hashcat/john.

3.4 — NetworkMiner et autres outils complémentaires

NetworkMiner : Extrait automatiquement les fichiers, images, emails, credentials d'un PCAP. Interface graphique. Idéal pour un survol rapide.

Zeek (Bro) : Analyse passive, génère des logs (conn.log, http.log, dns.log). Excellent pour analyser de gros PCAP.

Suricata : IDS/IPS open source. Peut analyser un PCAP offline et détecter des signatures d'attaque.

tcpdump : CLI léger. Filtre BPF identique aux Capture Filters Wireshark. Idéal pour capturer sur un serveur distant.

ngrep : grep sur des paquets réseau. Cherche des patterns ASCII/regex dans le payload.

strings + grep : Sur un fichier PCAP brut : `strings capture.pcap | grep -i 'flag|password|CTF'`

■ PLATEFORMES PRATIQUES — JOUR 3

- **CyberDefenders — BlueteamLabs** : challenges "PacketMaze", "Tomcat Takeover"
- **TryHackMe — Carnage** : analyse de trafic C2 réaliste
- Télécharge des PCAPs sur malware-traffic-analysis.net et essaie d'identifier les IOCs
- Lance un **ARP poisoning** dans un lab (Kali + VM cible) et analyse la capture

JOUR 4

CRYPTOGRAPHIE RÉSEAU — TLS, SSH, VPN

4.1 — TLS/SSL : comprendre le handshake

TLS chiffre les données mais le **handshake** lui-même est en clair et très informatif. Tu peux extraire : la version TLS, les cipher suites proposées, le certificat, le nom du serveur (SNI).

```
-- Types de messages TLS handshake
tls.handshake.type == 1 -- Client Hello
tls.handshake.type == 2 -- Server Hello
tls.handshake.type == 11 -- Certificate
tls.handshake.type == 12 -- Server Key Exchange
tls.handshake.type == 14 -- Server Hello Done
tls.handshake.type == 16 -- Client Key Exchange
tls.handshake.type == 20 -- Finished

-- Informations utiles
tls.handshake.extensions_server_name -- SNI (quel site le client visite)
tls.handshake.ciphersuite -- Cipher suite négociée
tls.record.version -- Version TLS (0x0303 = TLS 1.2)

-- Alertes TLS (erreurs)
tls.alert_message.desc -- Code d'erreur
tls.alert_message.level == 2 -- Fatal error
```

Déchiffrer TLS dans Wireshark avec une clé RSA ou SSLKEYLOGFILE

Si le CTF te fournit la **clé privée RSA** du serveur (ou un fichier SSLKEYLOGFILE), Wireshark peut déchiffrer tout le trafic TLS :

```
-- Clé privée RSA (uniquement si pas de Perfect Forward Secrecy)
Edit > Preferences > Protocols > TLS > RSA keys list
Ajouter : IP serveur, port, protocol=http, Key File = server.key

-- SSLKEYLOGFILE (méthode moderne, fonctionne avec PFS)
export SSLKEYLOGFILE=~/sslkeys.log
firefox & # ou chrome --ssl-key-log-file=~/sslkeys.log
# Dans Wireshark : Edit > Preferences > Protocols > TLS > (Pre)-Master-Secret log filename
```

4.2 — SSH : analyse du handshake

Le contenu SSH est chiffré mais tu peux analyser le handshake pour identifier la version, les algorithmes, et détecter des connexions bruteforce.

```
-- Voir le trafic SSH
ssh

-- Version du serveur/client
ssh.protocol contains "OpenSSH"
```

```
-- Détection bruteforce : beaucoup de connexions TCP sur port 22
tcp.dstport == 22 and tcp.flags.syn == 1
-- Connexions rejetées rapidement = bruteforce
tcp.dstport == 22 and tcp.flags.reset == 1
```

4.3 — Certificats X.509 et informations extractibles

Les certificats TLS contiennent des **métadonnées importantes** et passent en clair pendant le handshake :

```
-- Champs du certificat
x509sat.UTF8String -- Valeurs texte du certificat
x509ce.dNSName -- Subject Alternative Names (domaines)
tls.handshake.certificate -- Le certificat brut
-- En CTF : le flag peut être dans le CN ou SAN du certificat !
tls.handshake.extensions.server_name -- SNI
-- Exporter un certificat
-- Clic droit sur le paquet Certificate > Export Packet Bytes
-- Puis : openssl x509 -inform der -in cert.der -text
```

■ PLATEFORMES PRATIQUES — JOUR 4

- **PicoCTF** — "Trivial Flag Transfer Protocol", "b1g br0th3r"
- **CyberDefenders** — Challenges impliquant TLS avec clé fournie
- Pratique : génère un certificat self-signed, lance un serveur HTTPS, capture le trafic et analyse le handshake
- **TryHackMe** — **Protocols and Servers** pour consolider les bases

JOUR 5

FIREWALL, IDS/IPS & PIVOTING RÉSEAU

5.1 — Techniques d'évasion réseau

Les CTF avancés testent ta capacité à identifier des techniques d'évasion dans des captures. Voici les principales :

Fragmentation IP : Découpe les paquets pour passer sous les signatures IDS. Wireshark les réassemble automatiquement, mais note les offsets.

TTL Expiration : Envoie des paquets avec TTL qui expirent sur le firewall mais pas sur la cible (evasion de firewall stateless).

Protocol confusion : Encapsule du trafic dans un protocole autorisé (HTTP, DNS, ICMP tunneling).

Fragmentation TCP : PSH flag splitting, réordonnancement de segments pour tromper les IDS basés sur signatures.

Padding/Junk data : Ajoute des données inutiles pour décaler les signatures IDS.

5.2 — Tunneling : données dans des protocoles autorisés

Méthode	Filtre de base	Signe suspect	Outils
DNS Tunneling	dns	dns.qry.name.len > 40 or dns.qry.type == 16	iodine, dnscat2
ICMP Tunneling	icmp	icmp.type == 8 and data.len > 64	ptunnel, icmptunnel
HTTP Tunneling	http	http.request.method == "POST" and http contains "=="	Chisel, reGeorg
HTTPS Tunneling	tls	tls — analyser le SNI et la fréquence	Chisel, ngrok
SSH Tunneling	tcp.dstport==22	tcp.dstport == 22	SSH -L/-R/-D

5.3 — Pivoting : reconnaissance de la topologie réseau

Dans un CTF avec plusieurs machines, comprendre la topologie depuis une capture est crucial.

```
-- Identifier les passerelles (routers)
-- Un hôte qui forward du trafic avec une TTL décrémentée
-- Voir les subnets présents
-- Statistics > Endpoints > IPv4 (filtre par subnet)
-- Identifier les scans de découverte
icmp.type == 8 -- ping sweep
tcp.flags.syn == 1 and tcp.flags.ack == 0 -- SYN scan multi-ports
arp -- ARP sweep sur le LAN
-- Connexions inhabituelles (pivot)
-- Machine A parle à Machine B sur un port inhabituel
```

```
-- Machine B re-forward sur Machine C : voir les IPs sources/destinations
```

■ PLATEFORMES PRATIQUES — JOUR 5

- **Hack The Box — Sherlock's** : "Napper", "OpTinselTrace" (forensics réseau avancé)
- **CyberDefenders — Innerspace** : pivoting et tunneling
- Lab maison : configure un tunnel DNS avec iodine et capture le trafic pour analyser les patterns

JOUR 6

FORENSICS PCAP — Méthode CTF Complète

6.1 — Checklist systématique pour un challenge PCAP

- **Taille du fichier** : pcap > 100MB = Zeek/tshark plutôt que Wireshark GUI
- **Protocol Hierarchy** : Quels protocoles ? Proportions normales ?
- **Conversations** : Qui parle à qui ? Volumes suspects ?
- **Export Objects HTTP** : Fichiers transférés ? Images ? Archives ?
- **frame contains 'flag'** : Recherche directe du flag en clair
- **Credentials en clair** : FTP, HTTP Auth, Telnet — extraire les mots de passe
- **DNS exfil** : Noms longs, requêtes TXT, fréquences anormales
- **ICMP payload** : Data dans les pings — steganographie dans ICMP
- **TCP streams suspects** : Follow stream sur les connexions vers des ports non-standards
- **Fichiers binaires reconstitués** : Assembler des transferts FTP, SMB

6.2 — Reconstruction de fichiers depuis un PCAP

Méthode 1 : Export Objects (la plus simple)

```
-- Dans Wireshark GUI
File > Export Objects > HTTP (images, HTML, JS, etc.)
File > Export Objects > SMB (fichiers Windows partagés)
File > Export Objects > FTP-DATA (fichiers FTP)
File > Export Objects > DICOM (imagerie médicale)
File > Export Objects > TFTP (firmwares, configs)
```

Méthode 2 : Follow Stream + Save as raw bytes

1. Identifier le stream (Conversations ou filtre tcp.stream)
2. Clic droit > Follow > TCP Stream
3. Dans la fenêtre : Show data as "Raw" ou "Hex Dump"
4. "Save as..." > enregistrer les raw bytes
5. Identifier le type : file mystere_file
6. Ouvrir avec l'outil approprié

Méthode 3 : tshark + foremost/binwalk

```
# Extraire le payload d'un stream spécifique
tshark -r cap.pcap -q -z follow,tcp,raw,0 | xxd -r -p > stream0.bin

# Analyser avec binwalk (cherche magic bytes)
binwalk stream0.bin

# Extraire avec foremost (carving)
foremost -i cap.pcap -o ./output/

# Identifier le type de fichier
file stream0.bin
```

```
xxd stream0.bin | head -5 # magic bytes manuellement
```

Magic bytes à connaître

```
PNG : 89 50 4E 47 0D 0A 1A 0A
JPEG : FF D8 FF
PDF : 25 50 44 46 (%PDF)
ZIP : 50 4B 03 04 (PK)
GIF : 47 49 46 38 (GIF8)
ELF : 7F 45 4C 46 (.ELF)
PE/EXE : 4D 5A (MZ)
RAR : 52 61 72 21 (Rar!)
7z : 37 7A BC AF
```

6.3 — Steganographie dans les protocoles réseau

ICMP Payload Stego : Les requêtes ping contiennent des données cachées. Follow ICMP stream, extraire le payload, décoder.

DNS Subdomain Encoding : Les données sont encodées en base32/base64 dans les noms DNS. Extraire tous les qry.name, concaténer, décoder.

HTTP Header Stego : Des données cachées dans des headers HTTP custom ou dans des valeurs inhabituelles.

TCP Sequence Numbers : Le flag est encodé dans les numéros de séquence TCP initiaux (ISN). Extraire les ISN, décoder en ASCII.

IP ID Field : Données encodées dans le champ IP Identification. ip.id en hex → ASCII.

Timing Channel : Les intervalles entre paquets encodent des bits. Difficile mais possible.

■ PLATEFORMES PRATIQUES — JOUR 6

- **PicoCTF** — "Shark on Wire 1 & 2", "Trivial Flag Transfer Protocol", "Packets Primer"
- **CyberDefenders** — "PCAP Analysis", "Tomcat Takeover", "OpTinselTrace"
- **Root-Me** (root-me.org) — Section "Réseau" : environ 40 challenges PCAP de tous niveaux
- Objectif du jour : résoudre 3-5 challenges PCAP complets en appliquant la checklist

JOUR 7

PROTOCOLES EXOTIQUES & COVERT CHANNELS

7.1 — Protocoles industriels et IoT

Les CTF avancés incluent parfois des protocoles industriels ou IoT. Wireshark les supporte tous avec ses dissecteurs.

Modbus : Protocole SCADA industriel. Ports 502/TCP. modbus dans Wireshark. Lecture de registres, commandes d'actionneurs.

MQTT : IoT messaging. Port 1883/TCP (8883 TLS). mqtt dans Wireshark. Topics, messages pub/sub.

CoAP : IoT UDP-based REST. Port 5683/UDP. coap dans Wireshark.

DNP3 : SCADA/énergie. Port 20000/TCP. dnp3 dans Wireshark.

BACnet : Bâtiments automatisés. Port 47808/UDP. bacapp dans Wireshark.

SSDP/UPnP : Découverte réseau. Port 1900/UDP. ssdp dans Wireshark. Souvent utilisé pour pivot.

7.2 — IPv6 : spécificités et pièges

```
-- Filtres IPv6
ipv6 -- tout le trafic IPv6
ipv6.src == ::1 -- loopback IPv6
ipv6.dst == ff02::1 -- multicast all-nodes
-- ICMPv6 (Neighbor Discovery = ARP de IPv6)
icmpv6.type == 135 -- Neighbor Solicitation (= ARP Request)
icmpv6.type == 136 -- Neighbor Advertisement (= ARP Reply)
icmpv6.type == 134 -- Router Advertisement
-- Exfiltration via IPv6 tunnel dans IPv4 (6in4)
ip.proto == 41 -- protocole 41 = IPv6 encapsulé dans IPv4
```

7.3 — Covert Channels : dissimulation dans les en-têtes

Un covert channel utilise des champs normaux d'un protocole pour transmettre des données cachées. Voici comment les identifier :

IP Identification Field (ip.id) : ip.id varie de façon non-monotone. Extraire tous les ip.id en séquence.

TCP Initial Sequence Number (tcp.seq) : Premier paquet de chaque connexion : tcp.seq peut encoder un caractère ASCII.

TCP Urgent Pointer (tcp.urgent_pointer) : tcp.flags.urg == 1 — données urgentes peuvent cacher un canal.

IP Options (ip.opt) : Champs optionnels IP rarement utilisés — terrain fertile pour stego.

TCP Options / Timestamps (tcp.options.timestamp.tsval) : Valeurs de timestamp qui encodent des données.

Script tshark pour extraire les IP ID en séquence

```
# Extraire les ip.id de tous les paquets d'une source
```

```
tshark -r cap.pcap -Y "ip.src == 10.0.0.1" -T fields -e ip.id > ids.txt  
# Convertir en ASCII (Python)  
python3 -c "  
with open("ids.txt") as f:  
for line in f:  
val = int(line.strip(), 16)  
if 32 <= val <= 126:  
print(chr(val), end="")  
"
```

■ PLATEFORMES PRATIQUES — JOUR 7

- **Root-Me** — challenges réseau niveau avancé : "SNMP Authentication Failure", "Bluetooth Master"
- **CTFtime.org** — Archive des challenges réseau des CTF passés (télécharge les pcap)
- Challenge maison : crée un script Python qui encode un message dans les ip.id, puis décrypte-le depuis la capture

JOUR 8

SCRIPTING RÉSEAU — Scapy & Python

8.1 — Scapy : la boîte à outils Python réseau

Scapy permet de **créer, envoyer, capturer et analyser** des paquets réseau en Python. C'est l'outil parfait pour les challenges CTF qui demandent une interaction réseau programmatique.

Bases de Scapy

```
from scapy.all import *
# Lire un PCAP
pkts = rdpcap("capture.pcap")
# Itérer sur les paquets
for pkt in pkts:
    if pkt.haslayer(IP):
        print(pkt[IP].src, "->", pkt[IP].dst)
# Filtrer avec les layers
http_pkts = [p for p in pkts if p.haslayer(TCP) and p.haslayer(Raw)]
# Extraire le payload
for pkt in http_pkts:
    data = bytes(pkt[Raw].load)
    if b"flag" in data.lower():
        print(data)
```

Reconstruire des streams TCP avec Scapy

```
from scapy.all import *
from collections import defaultdict
def extract_tcp_streams(pcap_file):
    pkts = rdpcap(pcap_file)
    streams = defaultdict(bytes)
    for pkt in pkts:
        if pkt.haslayer(TCP) and pkt.haslayer(Raw):
            key = (pkt[IP].src, pkt[IP].dst,
                pkt[TCP].sport, pkt[TCP].dport)
            streams[key] += bytes(pkt[Raw].load)
    return streams
streams = extract_tcp_streams("capture.pcap")
for key, data in streams.items():
    print(f"Stream {key}: {len(data)} bytes")
    if b"flag" in data.lower():
        print("FLAG FOUND:", data)
```

8.2 — Scripts Python utiles pour les CTF

Décoder du DNS tunneling

```

import base64
from scapy.all import *

pkts = rdpcap("dns_tunnel.pcap")
data = ""

for pkt in pkts:
    if pkt.haslayer(DNS) and pkt[DNS].qr == 0: # requête
        name = pkt[DNS].qd.qname.decode()
        # Extraire le subdomain (avant le premier point)
        subdomain = name.split(".")[0]
        # Nettoyer et décoder base32
        try:
            decoded = base64.b32decode(subdomain.upper() + "=" * (8 - len(subdomain)%8))
            data += decoded.decode("utf-8", errors="ignore")
        except:
            pass
    print("Données extraites:", data)

```

Brute-force de challenge réseau avec socket

```

import socket

def send_and_receive(host, port, data):
    with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
        s.connect((host, port))
        s.sendall(data.encode())
        response = b""
        while True:
            chunk = s.recv(4096)
            if not chunk: break
            response += chunk
        return response.decode(errors="ignore")

# Exemple : challenge qui demande de résoudre des opérations
resp = send_and_receive("challenge.ctf", 1337, "hello\n")
print(resp)

```

Analyser les timing channels

```

from scapy.all import *

pkts = rdpcap("timing.pcap")
times = []

for i in range(1, len(pkts)):
    delta = pkts[i].time - pkts[i-1].time
    times.append(delta)

# Convertir les deltas en bits (ex: >0.1s = 1, <0.1s = 0)
bits = ["1" if t > 0.1 else "0" for t in times]
bitstring = "".join(bits)

# Décoder en ASCII (8 bits par caractère)
chars = [chr(int(bitstring[i:i+8], 2)) for i in range(0, len(bitstring), 8)]

```

```
print("Decoded:", "".join(chars))
```

■ PLATEFORMES PRATIQUES — JOUR 8

- **picoCTF** — challenges "scrambled-bytes", "packets primer" (nécessitent du scripting)
- **pwn.college** — modules réseau avec interaction programmatique
- **CTFtime archive** — Recherche "network scripting" dans les write-ups
- Exercice : écris un script qui reconstruit automatiquement tous les fichiers d'un PCAP HTTP

JOUR 9

SPRINT CTF PLATFORMS

9.1 — Plateformes recommandées et organisation

■ Root-Me	root-me.org	Réseau
Section Réseau avec ~40 challenges classés par difficulté. Très bien pour progresser. Commence par : Ethernet Frame, TELNET Authentication, FTP Authentication, PCAP IPv6		
■ PicoCTF	picoctf.org	All levels
Archive de challenges annuels. Cherche les tags 'forensics' + 'pcap'. Recommandés : Wireshark TWO, Packets Primer, WhatsThisSig		
■ CyberDefenders	cyberdefenders.org	Blue Team
Labs complets type SOC. Très réalistes. Enregistrement gratuit. Commence par : PacketMaze, HoneyBOT, PCAP Analysis		
■ TryHackMe	tryhackme.com	Guidé
Parcours guidés avec hints. Idéal si tu bloques. Rooms : Wireshark 101, Wireshark The Basics, h4cked, Carnage		
■ Hack The Box	hackthebox.com	Expert
Sherlocks challenges très réalistes. Niveau compétition. Commence par : Napper, Litter, Psittacosis		
■ CTFtime	ctftime.org	Archive
Archive des CTF passés. Télécharge les challenges réseau et lis les write-ups. Filtre par tag 'network' ou 'forensics'		

9.2 — Programme du Jour 9 (timing suggéré)

09h00	Root-Me	Résoudre 3 challenges réseau niveau 1-2 (Ethernet, FTP, HTTP)
11h00	PicoCTF	2 challenges PCAP de la section forensics
13h00	Pause	Lecture de write-ups sur CTFtime pour les challenges que tu n'as pas résolus
14h30	CyberDefenders	1 lab complet PacketMaze ou similaire
17h00	HTB Sherlocks	Tenter un Sherlock facile (Very Easy) lié au réseau
19h00	Bilan	Note tes lacunes, relis les sections correspondantes de ce cours

JOUR 10

SIMULATION COMPÉTITION COMPLÈTE

10.1 — Mindset CTF compétition

Un CTF n'est pas un examen académique. La stratégie compte autant que la technique.

- Commence par **trier tous les challenges** avant d'en résoudre un seul. Identifie les plus faciles.
- Un challenge non résolu en 30 minutes sans progression → **passe à autre chose**, reviens plus tard.
- Les challenges network/forensics ont souvent des **indices dans le nom et la description** du challenge.
- Google les outils et techniques spécifiques — en compét, tout est autorisé. **RTFM rapidement**.
- Lis les **premiers octets** de tout fichier suspect. Magic bytes = type de fichier.
- Si tu vois des données base64 : décode. Hex : convertis. Binaire : regroupe par 8.
- **Note tout** ce que tu trouves pendant l'analyse même si ça ne mène à rien immédiatement.

10.2 — Challenges de simulation (2 heures chrono)

Traite les challenges suivants en 2 heures, **sans aide** dans un premier temps. Note le temps par challenge. Compare ensuite avec les write-ups.

Niveau	Challenge	Temps cible	Compétence
Facile	Root-Me : FTP Authentication	10-15 min	Retrouver des credentials depuis un PCAP FTP
Facile	PicoCTF : Packets Primer	10-15 min	Extraction basique depuis PCAP
Moyen	Root-Me : OSPF Authentication	20-30 min	Protocole de routage dynamique, hash MD5
Moyen	CyberDefenders : PacketMaze	30-45 min	Multi-protocoles, lateral movement
Difficile	HTB Sherlock : au choix	45-60 min	Incident response réaliste

10.3 — Récapitulatif : ce que tu dois maîtriser après 10 jours

- Wireshark** : Filtres display arbitraires sans mémoriser • Follow streams • Export Objects • tshark CLI
- Protocoles** : TCP flags et handshake • DNS tunneling detection • HTTP credentials • TLS handshake metadata
- Forensics PCAP** : Checklist systématique • Reconstruction de fichiers • Magic bytes • Stéganographie réseau
- Scripting** : Scapy pour lire/analyser des PCAPs • Python pour automatiser le décodage • tshark pour l'extraction
- Mindset** : Triage rapide • Savoir quand passer à autre chose • Utiliser les indices dans les descriptions

■ TU ES PRÊT !

Après 10 jours d'entraînement intensif avec ce programme, tu possèdes une compréhension profonde des protocoles réseau, de Wireshark, et des méthodologies CTF. La clé : **tu comprends pourquoi** les filtres fonctionnent, tu ne les mémorises pas. Continue à pratiquer sur les plateformes, lis des write-ups, et participe à des CTF en ligne (CTFtime.org en liste de nouveaux chaque semaine). **Good luck et hack the planet ! ■**